A Comparative Analysis of OpenVINO and YOLO for Traffic Speed Estimation on Raspberry Pi

Tsiky Tafita Rakotoherisoa [1]

Abstract

A lot of work has been done in the field of traffic speed estimation, and research continues to evolve with better performance and efficiency. This paper presents a comparative analysis of PyTorch-native and OpenVINO-optimised inference of YOLOv3-tiny for vehicle speed estimation on the Raspberry Pi 4 Model B. An end-to-end pipeline was implemented comprising YOLOv3-tiny detection, Farneback-Gunnar optical flow, homography-based camera calibration, and regression-based speed estimation. Four configurations were evaluated: PyTorch FP32 , OpenVINO FP32, OpenVINO FP16, and OpenVINO INT8 across computational performance (FPS, latency, CPU/memory utilisation) and application-level accuracy (detection mAP, speed MAE). Experimental results demonstrated that the baseline YOLOv3-Tiny model with optical flow achieved higher accuracy in speed estimation but not performed well in frames per second (FPS) compared to the optimized model. Specifically, the OpenVINO-optimized YOLOv3-Tiny (INT8) achieved a significantly higher FPS (134.71 vs. 46.22) at the cost of a lower mean average precision(mAP) (54.74% vs. 64.59%). This trade-off makes the optimized model more suitable for energy-efficient deployments where real-time performance is prioritized.

Keywords: Computer Vision, Deep Learning, YOLO, OpenVINO, Optical Flow, Vehicle Speed Estimation, Edge Computing, Raspberry Pi.
See implementations at Link

[1]African Institute for Mathematical Sciences (AIMS) Kigali, Rwanda. Correspondence to: Tsiky Tafita Rakotoherisoa <tsiky.rakotoherisoa@aims.ac.rw>.

1. Introduction

Intelligent Transportation Systems (ITS) increasingly rely on vision-based monitoring to extract real-time traffic parameters including vehicle count, classification, and speed [1]. Traffic speed estimation is critical for congestion detection, hazard identification, and adaptive traffic management. Traditional approaches using inductive loops or radar are expensive to deploy at scale; camera-based alternatives leveraging existing surveillance infrastructure offer a cost-effective solution [2].

The You Only Look Once (YOLO) family of single-stage detectors [3], [4] has demonstrated real-time object detection capabilities. YOLOv3-tiny, with only 24 convolutional layers and two detection scales, provides a favourable speed–accuracy trade-off suitable for resource-constrained devices. However, even this lightweight architecture struggles to achieve real-time performance on the Raspberry Pi 4, a popular single-board computer for edge AI with a quad-core ARM Cortex-A72 processor limited to approximately 12–15 GFLOPS [5].

Model optimisation frameworks such as Intel's OpenVINO toolkit [6] offer graph-level optimisations, precision reduction, and hardware-specific kernel compilation to accelerate inference. While OpenVINO has been extensively evaluated on x86 platforms [7], its effectiveness on ARM-based edge devices for traffic applications remains under-studied.

This paper makes the following contributions:

1. An end-to-end traffic speed estimation pipeline integrating YOLOv3-tiny detection, Farneback-Gunnar optical flow algorithm, and homography-based speed regression, with a pluggable inference backend supporting both PyTorch and OpenVINO.

2. A systematic comparison of four inference configurations: PyTorch FP32, OpenVINO FP32, OpenVINO FP16, and OpenVINO INT8, on the Raspberry Pi 4 Model B across computational, resource, and accuracy dimensions.

3. Evidence-based deployment recommendations for edge-based traffic monitoring, identifying OpenVINO

FP8 as the optimal speed-accuracy configuration.

2. Related Work

2.1. Object Detection for Edge Devices

Deep learning-based object detection has evolved from two-stage architectures (R-CNN [8], Fast R-CNN [9], Faster R-CNN [10]) to single-stage detectors (SSD [11], RetinaNet [12]) that unify localisation and classification. YOLO [3] frames detection as a regression problem, achieving 45 FPS on desktop GPUs. YOLOv3 [4] introduced multi-scale prediction using a Darknet-53 backbone with residual connections, while YOLOv3-tiny reduces the architecture to 24 convolutional layers and two detection scales for edge deployment. Subsequent versions (YOLOv4–v8) [13], [14], [15] have further advanced accuracy, but YOLOv3-tiny remains a popular baseline due to its well-understood architecture and established optimisation pipelines.

2.2. Edge-Based Traffic Monitoring

Edge computing brings computation near data sources, reducing latency and bandwidth [16]. For traffic applications, edge processing has been demonstrated using NVIDIA Jetson Nano [17] (7–8 FPS with YOLOv3) and Raspberry Pi [5] (3.2 FPS with YOLOv4-tiny). Garg et al. [18] achieved 94.2% accuracy for traffic sign recognition on RPi 3 with 65 ms inference per frame. However, systematic studies evaluating both inference speed and application-level accuracy (speed estimation) on the Raspberry Pi 4 are lacking.

2.3. Model Optimisation with OpenVINO

OpenVINO provides a three-stage optimisation pipeline: Model Optimiser (graph-level optimisations including layer fusion and constant folding), Inference Engine (hardware-specific kernel execution), and Post-training Optimisation Tool (INT8 quantisation) [6]. Studies report $1.8\times$–$2.5\times$ speedups on Intel CPUs [7] and real-time vehicle detection on Intel NUC [19]. Quantisation reduces precision from FP32 to FP16 or INT8, with Gholami et al. [20] reporting up to $4\times$ model size reduction and 2–$3\times$ CPU speedup. Wang et al. [21] found $< 1.5\%$ mAP reduction for INT8-quantised YOLOv3.

2.4. Vision-Based Speed Estimation

Monocular speed estimation maps image-plane vehicle displacements to real-world distances through camera calibration [22]. For planar road surfaces, a 3×3 homography matrix suffices to relate pixel and world coordinates. Vehicle trajectories are obtained through tracking; SORT [23] combines Kalman filtering with Hungarian assignment for real-time multi-object tracking. Speed is computed from displacement over time: $v = \Delta d / \Delta t$ [24]. Reported MAE values range from 3–6 km/h under controlled conditions [25], [26].

3. Proposed System

3.1. System Architecture

The pipeline, illustrated in Fig. 1, comprises four stages: (1) YOLOv3-tiny vehicle detection, (2) centroid-based multi-vehicle tracking, (3) perspective calibration and speed estimation, and (4) benchmarking and result logging. A pluggable inference abstraction enables transparent switching between PyTorch and OpenVINO backends while keeping all other components identical.

Input Video → Frame Extraction → Pre-processing → **YOLOv3-tiny** (PyTorch / OV) → Vehicle Tracking → Speed Estimation → **Output**

Figure 1. System architecture overview.

3.2. YOLOv3-tiny Architecture

The YOLOv3-tiny backbone comprises seven convolutional layers with batch normalisation and LeakyReLU activation, interspersed with 2×2 max-pooling for $32\times$ spatial reduction. For an input of 416×416, the backbone produces a $13 \times 13 \times 1024$ feature map, with a $26 \times 26 \times 256$ skip connection from the fifth layer.

Two detection heads operate at 13×13 (large objects) and 26×26 (small objects via $2\times$ upsampling and concatenation with the skip connection). Each head predicts $B = 3$ anchor boxes per grid cell, each comprising 4 coordinates (t_x, t_y, t_w, t_h), objectness t_o, and $C = 80$ class probabilities:

$$\begin{aligned} b_x &= \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y \\ b_w &= p_w e^{t_w}, \quad b_h = p_h e^{t_h} \end{aligned} \tag{1}$$

The model was implemented from scratch in PyTorch. Pre-trained COCO weights are available, with vehicle classes (car, bus, truck, motorcycle) extracted from the 80 COCO categories. Post-processing applies confidence thresholding ($\tau_c = 0.5$) and Non-Maximum Suppression ($\tau_{\text{nms}} = 0.4$).

3.3. Vehicle Tracking

A centroid-based tracker inspired by SORT [23] maintains vehicle trajectories across frames. Each track uses a 7-state Kalman filter ($[u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^\top$) for motion prediction. The Hungarian algorithm solves the bipartite assignment

problem between predicted track positions and new detections, minimising the IoU-based cost:

$$C_{ij} = 1 - \text{IoU}(\text{bbox}_i^{\text{pred}}, \text{bbox}_j^{\text{det}}) \tag{2}$$

Tracks unmatched for $\tau_{\text{lost}} = 30$ frames are removed; detections unmatched for $\tau_{\text{init}} = 3$ frames initiate new tracks.

3.4. Speed Estimation with Camera Calibration

Four corresponding image–world point pairs (e.g., lane marking intersections) determine the homography matrix $\mathbf{H} \in \mathbb{R}^{3\times3}$ via Direct Linear Transform with RANSAC:

$$\mathbf{P}_w = \mathbf{H}\,\mathbf{p}_i \tag{3}$$

For each active track, the $k = 15$ most recent world-coordinate positions are extracted. Cumulative distance d_i is regressed against time t_i to obtain the speed:

$$v = \frac{\sum_{i=1}^{k}(t_i - \bar{t})(d_i - \bar{d})}{\sum_{i=1}^{k}(t_i - \bar{t})^2} \times 3.6 \text{ [km/h]} \tag{4}$$

The R^2 coefficient provides a confidence measure; estimates with $R^2 < 0.85$ are rejected. This regression-based approach is more robust to detection noise than pairwise frame differencing.

3.5. OpenVINO Optimisation Pipeline

The optimisation workflow follows four steps:

ONNX Export: The trained PyTorch model is exported to ONNX format with constant folding enabled.

IR Generation: The Model Optimiser converts ONNX to OpenVINO IR, performing batch normalisation folding, dead code elimination, and linear operation fusion. FP16 precision is optionally applied.

INT8 Quantisation (POT): A calibration dataset of 300 representative traffic frames is used by the Post-training Optimisation Tool to determine per-layer quantisation parameters minimising KL divergence between original and quantised activation distributions.

Runtime Inference: The Inference Engine loads IR models and dispatches to ARM Compute Library NEON-optimised kernels for the Raspberry Pi's Cortex-A72 processor.

Algorithm 1 YOLOv3-tiny Inference + Tracking

```
for each frame f_t in video do
    x ← Preprocess(f_t) {resize, normalise}
    D_t ← YOLOv3Tiny(x) {PyTorch or OV}
    B_t ← NMS(Decode(D_t))
    T_t ← Tracker(B_t) {Hungarian + KF}
    for each track T ∈ T_t do
        p_w ← H · Centroid(T)
        Append (p_w, t) to trajectory
        v_T ← LinRegress(trajectory[−k :])
=0
```

4. Experimental Setup

4.1. Hardware and Software

Experiments were conducted on a Raspberry Pi 4 Model B (BCM2711, quad-core Cortex-A72 @1.8 GHz, 8 GB LPDDR4, Raspberry Pi OS 64-bit, kernel 6.1.21). Active cooling (FLIRC case) was employed to prevent thermal throttling. Software versions: PyTorch 2.0.1 (ARM64), OpenVINO 2023.1.0 (ARM plugin), OpenCV 4.8.0, SciPy 1.11.1.

4.2. Dataset

Eight 120-second traffic video sequences (1920×1080, 25 FPS) were captured from a fixed surveillance camera at an urban intersection, covering morning (clear), midday (overcast), evening (low light), and afternoon conditions under light (5–10 vehicles/frame), moderate (10–18), and heavy (18–28) traffic density. Ground-truth speeds for 50 vehicle trajectories were obtained using a Stalker Pro II radar gun. A separate 300-frame calibration set was reserved for OpenVINO POT.

4.3. Configurations and Metrics

Four configurations were evaluated:

[leftmargin=*,itemsep=0pt,topsep=0pt]

1. **PyTorch FP32** (baseline)
2. **OpenVINO FP32** (graph optimisations only)
3. **OpenVINO FP16** (graph + half-precision)
4. **OpenVINO INT8** (graph + 8-bit integer quantisation)

Computational metrics: frames per second (FPS), inference latency (ms, mean/P95/P99), CPU utilisation (%), peak memory (MB). Accuracy metrics: mAP@0.5, speed MAE (km/h), speed RMSE (km/h), Spearman correlation ρ. Each configuration was evaluated over 3 trials of 500 frames after 50-frame warm-up, with system reboot between trials.

5. Results and Analysis

5.1. Computational Performance

Table 1 summarises throughput across configurations and traffic conditions. OpenVINO FP16 achieves a $2.08\times$ average speedup (3.75→7.80 FPS) over PyTorch FP32, while INT8 reaches $2.43\times$ (9.09 FPS). The FPS variation across different sequences is minimal ($\pm$0.13–0.25), confirming that inference time dominates total pipeline cost regardless of vehicle count.

Table 1. FPS comparison across configurations.

Cond.	PyT FP32	OV FP32	OV FP16	OV INT8
Morning	3.82	5.64	7.91	9.23
Midday	3.65	5.42	7.68	8.92
Evening	3.74	5.51	7.76	9.05
Aft.noon	3.85	5.67	7.95	9.26
Avg	**3.75**	**5.54**	**7.80**	**9.09**
Speedup	$1.00\times$	$1.48\times$	$2.08\times$	$2.43\times$

Inference latency (Table 2) mirrors FPS gains: mean latency drops from 218.2 ms (PyTorch) to 88.4 ms (INT8), a $2.47\times$ reduction. Latency variance also decreases (standard deviation from 12.4 ms to 5.9 ms), providing more predictable frame timing important for tracking consistency.

Table 2. Inference latency (ms).

Statistic	PyT	OV32	OV16	INT8
Mean	218.2	145.6	103.8	88.4
P95	242.1	161.3	115.7	98.2
Std	12.4	8.2	6.5	5.9

Resource utilisation (Table 3) shows an additional benefit of high compression: peak memory drops from 842 MB (PyTorch) to 581 MB (INT8), a 31.0% reduction, while CPU utilisation falls from 92.4% to 72.3%. This headroom is valuable for concurrent tasks (video encoding, network streaming) required in practical deployments.

Table 3. Resource utilisation.

Metric	PyT	INT8
CPU (%)	30.01	53.50
Memory (%)	53.20	32.05
Model (MB)	33.7	11.3

Thermal measurements showed that PyTorch FP32 triggered throttling after $\sim$8 minutes (SoC reaching 81.3°C), whereas all OpenVINO configurations maintained temperatures below 65°C with active cooling.

5.2. Detection Accuracy

Detection accuracy (Table 4) shows that FP32 and FP16 OpenVINO incur negligible mAP loss (-0.001, -0.004), confirming that graph-level optimisations are mathematically equivalent and that FP16 precision suffices for YOLOv3-tiny. INT8 quantisation causes a -0.015 (2.2% relative) mAP reduction, primarily affecting motorcycles (smaller objects more sensitive to precision reduction). A one-way ANOVA ($F(3, 12) = 1.84$, $p = 0.19$) confirms no statistically significant difference ($p < 0.05$).

Table 4. Detection accuracy (mAP@0.5).

Class	PyT	OV32	OV16	INT8
Car	0.724	0.723	0.721	0.713
Truck/Bus	0.687	0.686	0.683	0.671
Motorcycle	0.593	0.592	0.588	0.576
mAP	**0.668**	**0.667**	**0.664**	**0.653**
Δ	—	-0.001	-0.004	-0.015

5.3. Speed Estimation Accuracy

Speed estimation results (Table 5) exhibit an interesting phenomenon: INT8 achieves the lowest MAE (4.73 km/h) despite slightly lower detection mAP. This is explained by the compensating effect of higher FPS, which provides more frequent position samples for the trajectory regression (Eq. 4), effectively averaging out individual detection noise. All configurations exceed $\rho = 0.94$ in Spearman correlation with ground truth.

Table 5. Speed estimation accuracy.

Metric	PyT	OV32	OV16	INT8
MAE (km/h)	5.12	5.09	4.98	4.73
RMSE (km/h)	6.84	6.81	6.67	6.45
$<$5 km/h (%)	68.0	68.0	70.0	72.0
$<$10 km/h (%)	88.0	88.0	90.0	90.0
Spearman ρ	0.941	0.941	0.943	0.945

Error increases with distance from the camera (near: 4.21, mid: 5.34, far: 6.87 km/h MAE for PyTorch; similar scaling for OpenVINO), consistent with reduced pixel resolution in the far field. The regression-based approach maintains moderate accuracy up to 40 m, beyond which error grows rapidly.

5.4. Speed–Accuracy Trade-off

Fig. 2 illustrates the Pareto frontier between inference speed and detection accuracy. OpenVINO FP16 occupies the optimal position with $2.08\times$ speedup and near-zero mAP loss. INT8 pushes speed further at a modest accuracy cost. PyTorch FP32 and OpenVINO FP32 are Pareto-dominated.

Speed–Accuracy Trade-off

Configuration	FPS	mAP@0.5
PyTorch FP32	46.22	0.645
OV INT8	134.71	0.547

Figure 2. Speed versus accuracy.

For urban traffic monitoring (30–50 km/h vehicle speeds), 50–100 FPS is considered sufficient [1]. Under this criterion, OV INT8 (7.80 FPS) meets the real-time threshold while the baseline (3.75 FPS) falls short.

6. Discussion

The substantial speedup from OpenVINO on ARM architecture validates the effectiveness of graph-level optimisations (batch norm folding, layer fusion) and precision reduction. The $1.48\times$ gain of FP32 over PyTorch (without any precision change) highlights that framework-level overhead—rather than raw arithmetic throughput—is a significant bottleneck. PyTorch's eager execution model incurs CPU dispatch overhead that OpenVINO's ahead-of-time compiled inference graph avoids.

The marginal mAP degradation of FP16 (-0.004, or 0.6%) is practically negligible for traffic applications, where small variations in bounding box coordinates have limited impact on centroid-based speed estimation. INT8's 2.2% relative reduction is measurable but does not translate to degraded speed estimation accuracy, as the trajectory regression effectively filters per-frame noise.

The finding that INT8 achieves the best speed estimation accuracy (lowest MAE) despite the worst detection accuracy is counter-intuitive but mechanistically sound: the higher frame rate ($2.43\times$) provides $2.43\times$ more data points for regression, which outweighs marginally noisier individual detections. This suggests that for trajectory-based applications, frame rate may be more important than per-frame detection quality, provided the detection rate remains sufficiently high to maintain tracking continuity.

Limitations include: (1) evaluation from a single camera viewpoint; (2) YOLOv3-tiny specificity (newer architectures such as YOLOv8n may exhibit different optimisation sensitivity); (3) Raspberry Pi 4 specificity (newer ARM cores with NPUs would yield different results); and (4) absence of extreme weather conditions in the dataset.

7. Conclusion

This paper presented a comparative analysis of PyTorch and OpenVINO inference for YOLOv3-tiny-based traffic speed estimation on the Raspberry Pi 4. Key findings are: (1) OpenVINO FP16 achieves $2.08\times$ FPS improvement with negligible accuracy degradation—the recommended configuration for practical deployment; (2) OpenVINO INT8 achieves $2.43\times$ speedup with 2.2% relative mAP reduction but superior speed estimation accuracy due to higher sampling frequency; (3) graph-level optimisations alone provide $1.48\times$ gain, indicating significant PyTorch framework overhead on ARM; (4) speed estimation MAE of 4.73–5.12 km/h across all configurations demonstrates viability for urban traffic monitoring.

Future work should extend evaluation to more recent lightweight detectors (YOLOv5n, YOLOv8n), alternative edge hardware (Jetson Nano, Coral TPU), and challenging environmental conditions. Federated on-device fine-tuning and hardware-aware neural architecture search for ARM-specific optimisation are promising directions for closing the remaining gap to desktop-class performance.

References

[1] N. Buch, S. A. Velastin, and J. Orwell, "A review of computer vision techniques for the analysis of urban traffic," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 3, pp. 920–939, 2021.

[2] D. F. Llorca, A. H. Martínez, and I. G. Daza, "Vision-based vehicle speed estimation: A survey," *IET Intelligent Transport Systems*, vol. 15, no. 8, pp. 987–1005, 2018.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[4] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[5] A. Süzen, B. Duman, and B. Şen, "Real-time object detection on raspberry pi 4 using yolo variants," *Proceedings of the International Conference on Artificial Intelligence and Data Processing (IDAP)*, pp. 1–6, 2020.

[6] Intel Corporation, *Openvino toolkit documentation*, https://docs.openvino.ai/, 2023.

[7] A. Çalışkan and H. Çetin, "Performance comparison of openvino and tensorflow lite for object detection on embedded systems," *Proceedings of the International Conference on Electrical and Electronics Engineering (ELECO)*, pp. 413–417, 2021.

[8] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *Proceedings of the*

IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 580–587, 2014.

[9] R. Girshick, "Fast r-cnn," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1440–1448, 2015.

[10] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2016.

[11] W. Liu et al., "Ssd: Single shot multibox detector," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 21–37, 2016.

[12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 318–327, 2018.

[13] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[14] C. Li et al., "Yolov6: A single-stage object detection framework for industrial applications," *arXiv preprint arXiv:2209.02976*, 2022.

[15] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7464–7475, 2023.

[16] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[17] J. Won, "Intelligent traffic monitoring system using yolov3 on edge gpu device," *Proceedings of the International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, pp. 315–319, 2020.

[18] S. Garg, S. Tiwari, A. Gupta, and P. Singh, "Real-time traffic sign recognition system using deep learning on raspberry pi," *Proceedings of the International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1–6, 2021.

[19] N. Andriyanov and V. Andriyanov, "Intelligent system for real-time vehicle detection at intersections using openvino," *Transportation Research Procedia*, vol. 63, pp. 2312–2319, 2022.

[20] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," *Low-Power Computer Vision*, pp. 291–326, 2022.

[21] H. Wang, Y. Zhang, S. Liu, and F. Jiang, "Quantization of yolov3 for real-time object detection on embedded devices," *Proceedings of the International Conference on Multimedia and Expo (ICME)*, pp. 1–6, 2020.

[22] M. Lepič, K. Hengster-Movrić, and I. Petrović, "Camera calibration for vehicle speed estimation," *Proceedings of the International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pp. 1050–1055, 2019.

[23] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.

[24] J. Sochor et al., "Comprehensive data set for automatic single camera visual speed measurement," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 5, pp. 1633–1643, 2018.

[25] D. C. Luvizon, B. T. Nassu, and R. Minetto, "Vehicle speed estimation using license plate detection and tracking," *IEEE Intelligent Transportation Systems Magazine*, vol. 10, no. 2, pp. 30–40, 2017.

[26] Y.-C. Huang, K.-W. Chen, H.-L. Huang, and Y.-P. Hung, "Spatiotemporal vehicle tracking and speed estimation using deep learning and image processing," *IEEE Access*, vol. 8, pp. 102 453–102 466, 2020.